Web API Introduction

Distributed Application Development in the Cloud- Net 031

Prepared By: Rupinder Sandhu
Rupinder.sandhu@humber.ca

HUMBER

WE ARE HUMBER

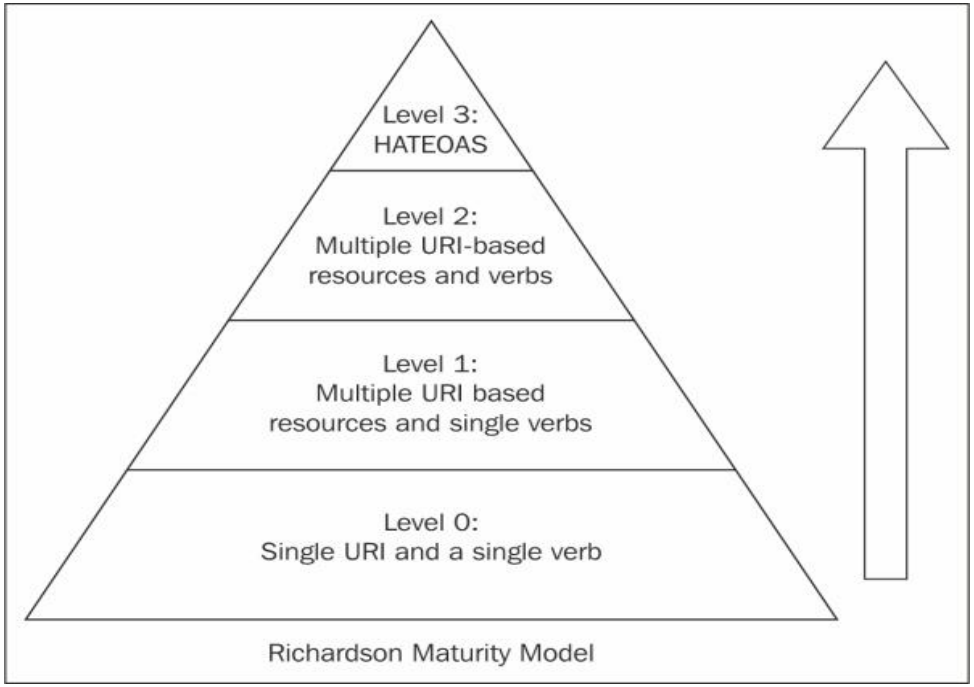
Agenda

- REST Overview
- Richardson Maturity Model
- 6 Constraints of REST
- Advantages of using Web API

REST

- Representational State Transfer (REST)
- Created by Roy Fielding, one of the primary authors of the HTTP specification
- REST is meant to take better advantage of standards and technologies within HTTP than SOAP does today
- Rather than creating arbitrary SOAP methods, developers of REST APIs are encouraged to use only HTTP verbs (GET, POST, PUT, DELETE)

Richardson Maturity Model



https://restfulapi.net/richardson-maturity-model/

Level Zero

These services have a single URI and use a single HTTP method (typically POST). For example, most Web Services (WS-*)-based services use a single URI to identify an endpoint, and HTTP POST to transfer SOAP-based payloads, effectively ignoring the rest of the HTTP verbs

https://restfulapi.net/richardson-maturity-model/

Level One

These services employ many URIs but only a single HTTP verb – generally HTTP POST. They give each resource in their universe a URI. A unique URI separately identifies one unique resource – and that makes them better than level zero.

https://restfulapi.net/richardson-maturity-model/

Level Two

Level two services host numerous URI-addressable resources. Such services support several of the HTTP verbs on each exposed resource – Create, Read, Update and Delete (CRUD) services. Here the state of resources, typically representing business entities, can be manipulated over the network.

https://restfulapi.net/richardson-maturity-model/

Level Three

Level three of maturity makes use of all three, i.e. URIs and HTTP and HATEOAS.
This level is the most mature level of Richardson's model, which encourages easy discoverability. This level makes it easy for the responses to be self-explanatory by using HATEOAS.
The service leads consumers through a trail of resources, causing application state transitions as a result.

https://restfulapi.net/richardson-maturity-model/

HATEOAS

- Short for: Hypermedia as the Engine of Application State
- With HATEOAS, a client interacts with a network application whose application servers provide information dynamically through hypermedia
- A REST client needs little to no prior knowledge about how to interact with an application or server beyond a generic understanding of hypermedia
- Clients deliver state via body contents, query-string parameters, request headers and the requested URI
- Services deliver state to clients via body content, response codes, and response headers

6 Constraints of REST

- Uniform Interface
- Stateless
- Cacheable
- Client-Server
- Layered System
- Code on Demand (Optional)

Uniform Interface

- Defines the interface between clients and servers
- Resource-based
  - Individual resources are identified in requests using URIs as resource identifiers
- Self-descriptive messages
  - Each message includes enough information to describe how to process the message

Stateless

- The necessary state to handle the request is contained within the request itself
  - as part of the URI, query-string parameters, body, or headers.
- The client must include all information for the server to fulfill the request
  - resending state as necessary if that state must span multiple requests.
- Statelessness enables greater scalability since the server does not have to maintain, update or communicate that session state.

Cacheable

- In REST, caching shall be applied to resources when applicable, and then these resources MUST declare themselves cacheable
- Caching can be implemented on the server or client-side.
- Responses must, implicitly or explicitly, define themselves as cacheable, or not, to prevent clients reusing stale or inappropriate data in response to further requests

Client-Server

- Client application and server application MUST be able to evolve separately without any dependency on each other
- A client should know only resource URIs, and that's all
- The uniform interface separates clients from servers
  - Clients are not concerned with data storage
  - Servers are not concerned with the user interface or user state
  - Servers and clients may be replaced and developed independently, as long as the interface is not altered.

Layered System

- A client cannot tell whether it is connected directly to the end server, or to an intermediary along the way.
- Intermediary servers may improve system scalability by enabling load-balancing and by providing shared caches.

Code on Demand (Optional)

- Code-on-Demand (COD) is the only optional constraint in REST.
- It allows clients to improve its flexibility because in fact it is the server who decides how certain things will be done.
- With Code-On-Demand, a client can download a JavaScript in order to perform some action (e.g. encrypt communication)
  - Server is free to return an executable

Advantages of using Web API

- Built-in serialization
- Self-hosting
- Built-in dependency injection
- Native attribute routing
- Does not limit to use specific interface or technology
- Open Source
- It uses low bandwidth
- Web API Controller pattern is much similar to MVC Controller.
- Easy to test
- Lightweight architecture

THANK YOU.

HUMBER

WE ARE HUMBER